A ARKA-Lite

EQUIPO ARKA-LITOS · UPSKILLING BACKEND JAVA

Del endpoint al clúster

Cómo construimos dos microservicios con sus datos, su plataforma y su operación.

2 microservicios	**2** bases PostgreSQL	**1** comando levanta todo	**0** avisos perdidos o duplicados

VERSIÓN RESUMEN · STAFF DE TECNOLOGÍA



Qué vamos a ver

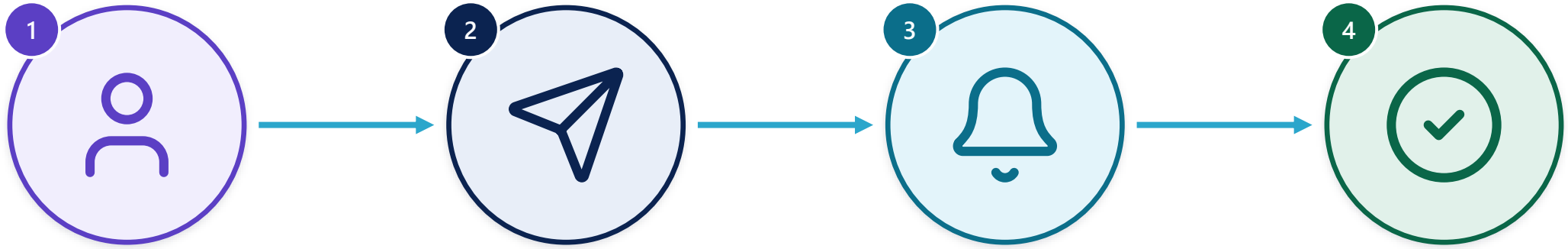
Las mismas secciones que todos los equipos; nosotros vamos a fondo en datos, interconexión y DevOps.



1	2	3	4	5	6	7
Fundamentos	**Arquitectura**	**Base de datos**	**Interconexión de servicios**	**DevOps: Docker y Kubernetes**	**Observabilidad**	**Demostración con la UI**
VISTA GENERAL	VISTA GENERAL	A FONDO	A FONDO	A FONDO	VISTA GENERAL	EN VIVO
Primero el qué · Código mantenible · Git y documentación	Microservicios · Una base por servicio · Hexagonal en una frase	Modelo de datos · ORM con JPA · Cadena de conexión	Síncrono y asíncrono · Patrón Outbox · Idempotencia	Docker y Compose · CI/CD con GitHub Actions · Kubernetes	Logs centralizados · Trazas · Salud con Actuator	Crear y enviar · Una caída en vivo · Reentregar

Primero el qué: el negocio en una lámina

Antes de escribir código, entendimos qué tiene que pasar, quién lo hace y qué no se permite.



Quien solicita

Registra qué necesita y de qué tipo.

La envía

Cuando está lista.

El sistema avisa

A quien debe atenderla.

Alguien decide

La aprueba o la rechaza.

si se salta un paso

No se permite

Enviar dos veces o decidir sobre un borrador: el sistema explica por qué.

Código mantenible: buenas prácticas

Después, el cómo: seis prácticas que hacen que un cambio toque un solo lugar, en cualquier lenguaje.

Una tarea por pieza

Quien recibe la petición no decide; quien decide no guarda.

Piezas que se conectan

Cada pieza recibe lo que necesita: se cambia o se prueba sola.

Reglas en un solo lugar

El negocio vive junto, lejos de la pantalla y de la base.

Errores con respuesta clara

Siempre el mismo formato: 400, 404 o 409 con su motivo.

Nombres del negocio

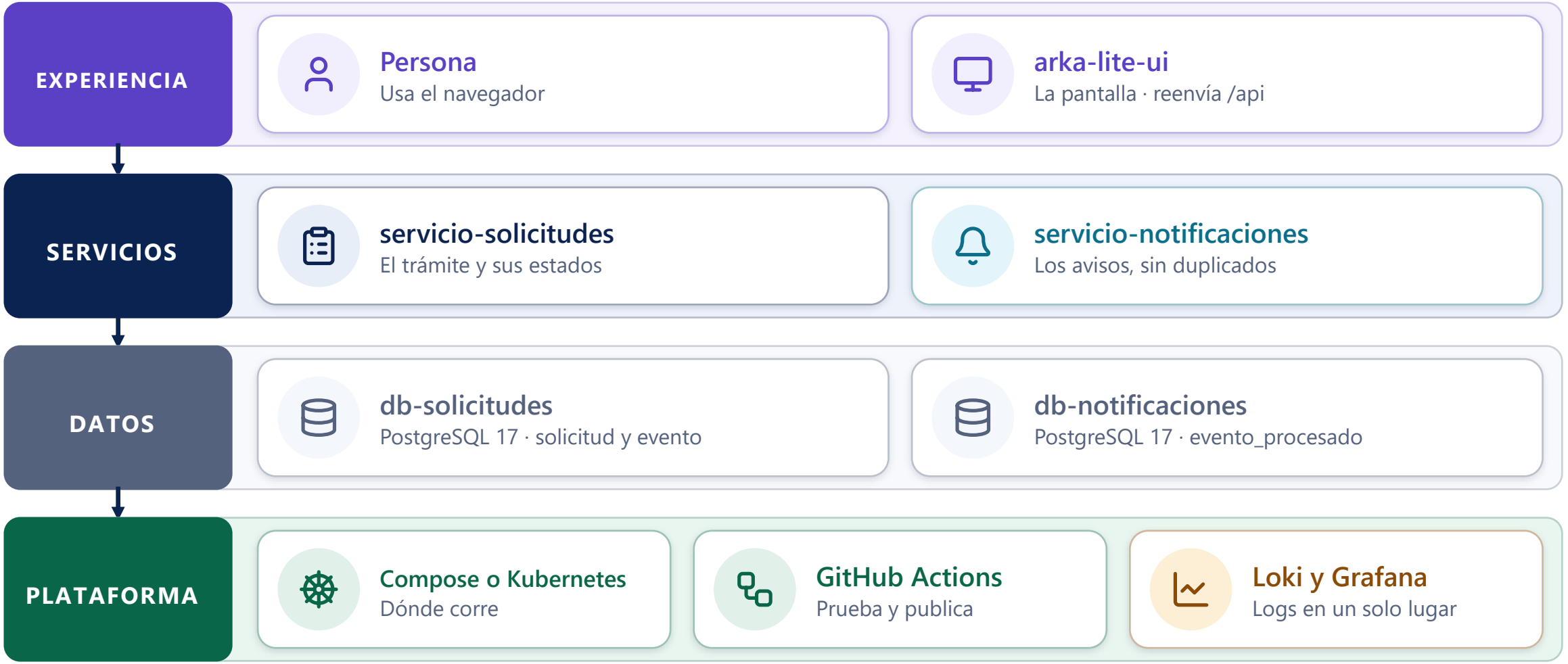
Solicitud, enviar, aprobar: el código se lee como el proceso.

Cada regla con su prueba

Una prueba automática avisa si un cambio rompió algo.

El sistema de un vistazo

Cada capa tiene una sola responsabilidad.



Cómo funciona en la práctica

Seis conexiones, y ninguna une las dos bases.



Ninguna flecha une las dos bases: cada servicio es dueño de sus datos.

Dos bases, una por servicio

Cada microservicio tiene su propia base, y las dos corren en PostgreSQL.

2

bases de datos

db-solicitudes y db-notificaciones: una por servicio.

17

PostgreSQL

El mismo motor y la misma versión en las dos.

0

JOIN entre bases

Ni consultas ni llaves foráneas que crucen de una a otra.

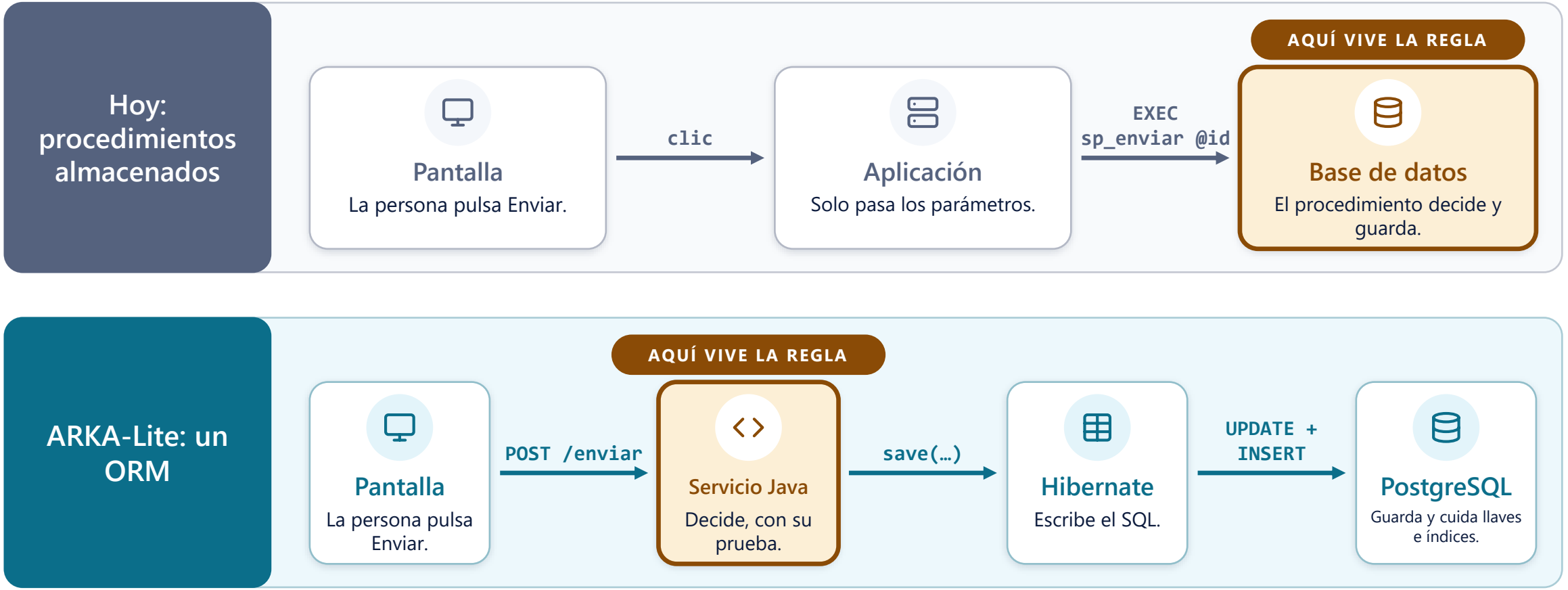
1

disco por base

En Kubernetes, los datos sobreviven aunque el contenedor se reinicie.

Del procedimiento almacenado al ORM

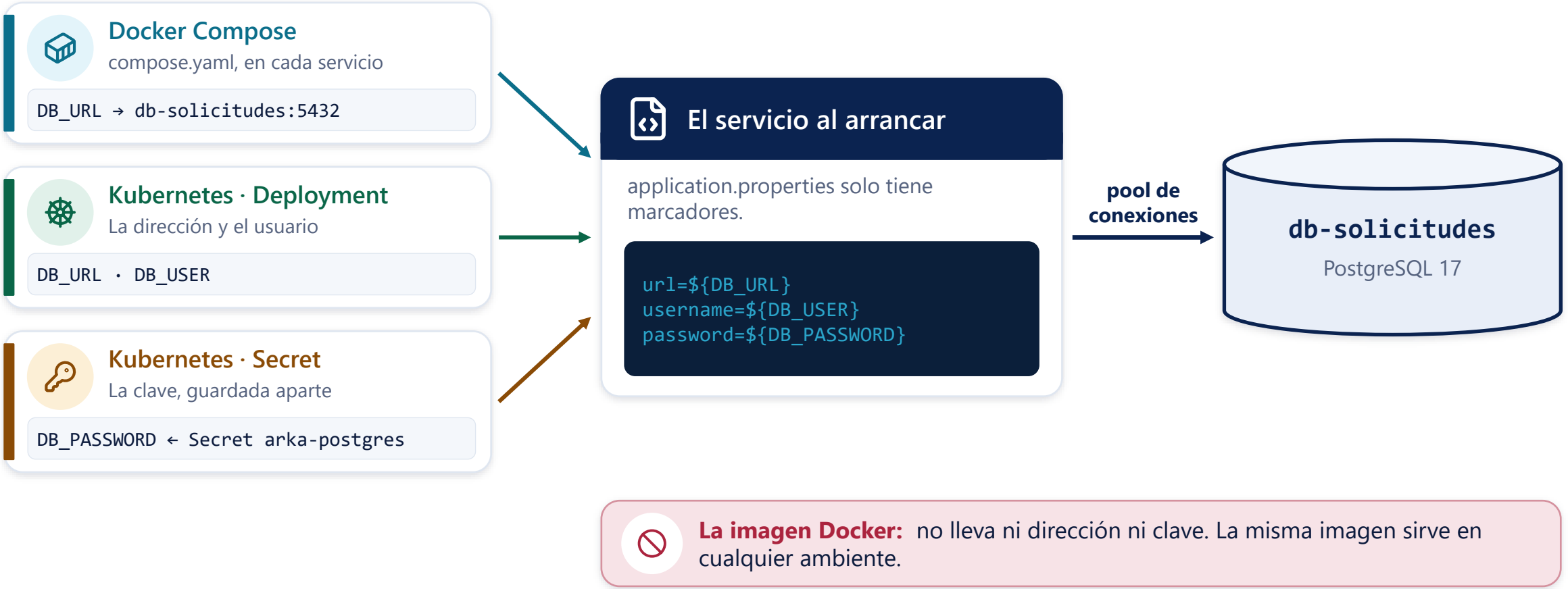
La diferencia no es la base de datos: es dónde vive la regla del negocio.



¿Por qué importa? No es todo o nada: las reglas van al código y los reportes o consultas pesadas pueden seguir en SQL.

¿Dónde está la cadena de conexión?

Ni en el código ni en la imagen: la pone el ambiente donde corre la aplicación.



¿Por qué importa? Cambiar de servidor o rotar una clave no obliga a recompilar ni a tocar el código.

Síncrono o asíncrono

Lo que la persona necesita ver se responde ya; lo que puede esperar se avisa después.

Síncrono

- La UI llama y espera que se complete el proceso.
- Respuesta inmediata
- Si el servicio se cae, la persona ve error

vs

Asíncrono

- Servicio solicitudes envía evento a servicio notificaciones y continúa con su proceso
- Respuesta puede demorar
- Si notificaciones se cae, el evento espera

Piénsenlo así: Síncrono es una llamada: me quedo en la línea hasta que me responden. Asíncrono es un correo: lo envío y sigo con lo mío.

Patrón Outbox: guardar primero, avisar después

La solicitud y su evento se guardan en la misma transacción; el relevador avisa después.



Enviar

Solicitud ENVIADA + evento pendiente, en una transacción.

Bandeja de salida

El evento espera en la tabla evento.

Relevador

Cada 5 s entrega los pendientes, hasta 3 intentos.

si falla

Sigue pendiente

Mismo eventoId; se reintenta en el siguiente ciclo.

Notificaciones

Lo recibe y el evento queda procesado.

Piénsenlo así: Como dejar una carta en la bandeja de salida: aunque el mensajero no pase hoy, la carta no se pierde.

¿Por qué importa? Si notificaciones está caído, ningún aviso se pierde: queda pendiente hasta que vuelve.

Docker y Kubernetes: no compiten, se complementan

Docker construye la caja; Kubernetes la pone a correr y la cuida.

Docker: empaqueta

- Mete la app con Java y todo lo que usa
- Crea una imagen que no cambia
- Corre igual en una laptop o en un servidor
- Compose levanta varias cajas juntas

+

Kubernetes: administra

- Decide dónde corre cada caja
- Si una se cae, levanta otra
- Le da a cada servicio un nombre fijo
- Puede correr más copias si hay carga

Piénsenlo así: Como el transporte de carga: Docker es el contenedor estándar y Kubernetes, el puerto que los acomoda, los vigila y reemplaza el que se daña.

Cuando un Pod se cae

Kubernetes vuelve solo al estado que declaramos.

QUÉ FALLA

Se cae el Pod de solicitudes Un error, un nodo reiniciado o un kubectl delete.



Detecta

Falta 1 de 1 copias.

Recrea

Un Pod nuevo con la misma imagen.

Espera

initContainers: base y notificaciones.

Listo

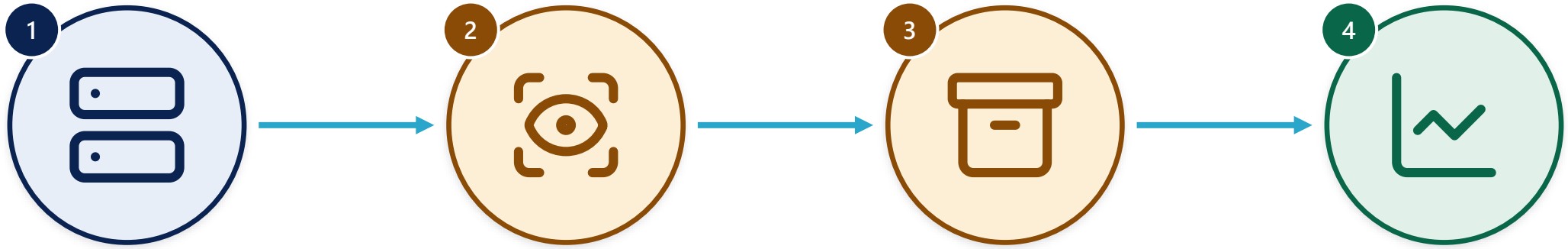
La readinessProbe responde.

RESULTADO

Sin intervención manual Los eventos pendientes siguen en la base y se entregan.

Todos los logs, en un solo lugar

Seguir un evento de un servicio a otro sin entrar a ningún contenedor.



Servicios

Logs JSON con traceId y eventId.

Promtail

Los recoge de cada contenedor.

Loki

Los guarda, etiquetados por servicio.

Grafana

Una búsqueda para todo el sistema.

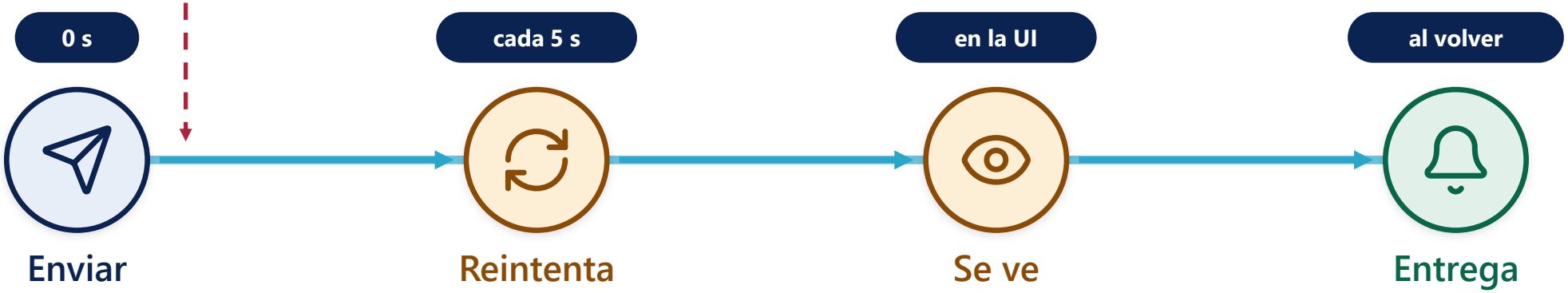
Piénsenlo así: Como la caja negra de un avión: registra lo que pasó, para revisarlo después.

En vivo: si notificaciones se cae

La persona no se entera; el aviso llega cuando el servicio vuelve, una sola vez.

QUÉ FALLA

notificaciones está caído docker compose stop servicio-notificaciones



Responde bien: solicitud ENVIADA y evento guardado.

3 intentos por ciclo; el evento sigue pendiente.

“Notificaciones no responde” y 1 en camino.

Mismo eventId: se crea el aviso.

RESULTADO

1 aviso, 0 perdidos Sin intervención y, al reentregar, sin duplicados.

El valor agregado

Prácticas que un equipo de tecnología puede reutilizar, en cualquier lenguaje.

PILAR	QUÉ CONSTRUIMOS	VALOR PARA EL EQUIPO
Datos consistentes	Transacción local, Outbox e idempotencia	0 avisos perdidos o duplicados ante caídas
Conexión segura	Cadena de conexión y claves fuera del código	Cambiar de servidor o de clave sin recompilar
Reproducibilidad	Docker, Compose y Kubernetes	1 comando y la misma imagen en todo ambiente
Calidad	Pruebas automáticas y pipeline en cada cambio	Errores detectados en minutos, antes de integrar
Trazabilidad	Logs centralizados, traceId e imagen por commit	Diagnóstico en minutos y versiones rastreables

Lo que pueden aplicar desde mañana

No hace falta reescribir nada: estas prácticas funcionan también con procedimientos almacenados.

PRÁCTICA	CÓMO SE VE EN SUS SISTEMAS DE HOY	QUÉ EVITA
Cadena de conexión fuera del código	Variables de entorno o un gestor de secretos en el servidor	Claves en el repositorio y recompilar por cambiar de servidor
Guardar y avisar en la misma transacción	Una tabla de pendientes que llena el procedimiento y un job que la entrega	Avisos perdidos cuando el otro sistema está caído
Idempotencia en lo que se reintenta	Una tabla de procesados con llave única por número de operación	Pagos, correos o registros duplicados
Un identificador que cruce sistemas	El mismo número en el log de la aplicación, del procedimiento y del otro sistema	Horas buscando un error entre sistemas
El mismo paquete en todos los ambientes	Contenedores para aplicaciones y bases de prueba	El clásico “en mi máquina funciona”

Lo que nos llevamos

LO QUE APRENDIMOS

0

avisos perdidos o duplicados

1

comando para levantar todo

Diseñar para la falla

Guardar primero, reintentar e ignorar duplicados.

VALOR

Sistemas que se recuperan solos.

Automatizar la confianza

Pruebas, pipeline, imágenes y estado declarado.

VALOR

Cambiar rápido sin miedo a romper.

Hacer visible el sistema

Logs, trazas, salud, Swagger y la UI.

VALOR

Diagnosticar y explicar en minutos.

A ARKA-Lite

Gracias

Equipo arka-litos

Proyecto final · Upskilling Backend Java

